FastAPI

Running FastAPI with Uvicorn

Quick Recap

- What is FastAPI
- FastAPI vs Flask vs Django
- ASGI vs WSGI
- First GET API
- Running app

Quick Quiz

- 1. Which command is used to run FastAPI?
- 2. Which decorator is used for GET API?
- 3. FastAPI is based on which standard? (ASGI/WSGI)
- 4. Which library is used to run FastAPI server?
- 5. What URL is used to access FastAPI by default?

ASGI vs WSGI

- **WSGI (Older):** Synchronous and blocking; handles one request at a time.
- **ASGI (Modern):** Asynchronous and non-blocking; allows handling multiple concurrent requests (e.g., WebSockets, long-polling).

FastAPI program

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def root():
    return {"message": "Hello World"}
```

Save & Run the file

- Save the file **: main.py**
- Run the live server **: fastapi dev main.py**
- **In the output, there's a line with something like:**

- INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)

- **That line shows the URL where your app is being served, in your local machine.**

On Browser

- Open your browser at **http://127.0.0.1:8000.**
- You will see the JSON response as: **{"message": "Hello World"}**

Interactive API docs

- http://127.0.0.1:8000/docs
- http://127.0.0.1:8000/redocs

Open API - Schema

- FastAPI generates a "**schema**" with all your API using the OpenAPI standard for defining APIs.

Open API - Schema

- **Schema :**
  - A "schema" is a definition or description of something.
  - Not the code that implements it, but just an abstract description.

Open API - Schema

- **API schema :**
  - In this case, OpenAPI is a specification that dictates how to define a schema of your API.
  - This schema definition includes your API paths, the possible parameters they take, etc.

Open API - Schema

- **Data schema :**
  - The term "schema" might also refer to the shape of some data, like a JSON content.
  - In that case, it would mean the JSON attributes, and data types they have, etc.

Open API - Schema

- **OpenAPI and JSON schema :**
  - OpenAPI defines an API schema for your API. And that schema includes definitions (or "schemas") of the data sent and received by your API using JSON Schema, the standard for JSON data schemas.

openapi.json

- FastAPI automatically generates a JSON (schema) with the descriptions of all your API.
- You can see it directly at: http://127.0.0.1:8000/openapi.json.
- It shows how the raw OpenAPI schema looks like :

Deploy your app

- We can optionally deploy our **FastAPI app** to **FastAPI Cloud (https://fastapicloud.com/).**
- Deploy the application with one command : fastapi deploy
- Before deploying, make sure you are logged in: fastapi login

Step by Step program explanation

- from fastapi import FastAPI

- app = FastAPI()

- @app.get("/")
- async def root():
- return {"message": "Hello World"}

Step – 1 : import FastAPI

- from fastapi import FastAPI
- app = FastAPI()
- @app.get("/")
- async def root():
- return {"message": "Hello World"}

- **FastAPI** is a **Python class** that provides all the (ASGI) functionality for API.
- Technical Details
- FastAPI is a class that inherits directly from Starlette (ASGI framework).
- For more details : https://www.starlette.dev/

Step – 2 :create a FastAPI "instance"

- from fastapi import FastAPI
- app = FastAPI()
- @app.get("/")
- async def root():
- return {"message": "Hello World"}

- Here the **app** variable will be an "instance" of the class **FastAPI.**
- This will be the main point of interaction to create all the API.

Step – 3.1 : create a path operation

- **Path :** refers to the last part of the URL starting from the first /.
- So, in a URL like: https://example.com/items/foo
- ...the path would be: /items/foo

- While building an API, the "path" is the main way to separate "concerns" and "resources".

- **Note :** A "path" is also commonly called an "endpoint" or a "route".

Step – 3.2 : create a path operation

- **Operation :** refers to one of the HTTP "methods".
- **Http methods** : POST, GET, PUT, DELETE, OPTIONS, HEAD, PATCH, TRACE
- In the HTTP protocol, you can communicate to each path using one (or more) of these "methods".
- When building APIs, you normally use these specific HTTP methods to perform a specific action.

Method	Operation
POST	To create data
GET	To read data
PUT	To update data
DELETE	To delete data

- Note: In OpenAPI, each of the HTTP methods is called an "operation".

Step – 3.3 : create a path operation

- from fastapi import FastAPI
- app = FastAPI()
- @app.get("/")
- async def root():
- return {"message": "Hello World"}

- The **@app.get("/")** tells FastAPI that the function right below is in charge of handling requests that go to:
- the **path /**
- using a **get operation**

Step – 3 : Info

- Other operations:

- @app.post()
- @app.put()
- @app.delete()
- @app.options()
- @app.head()
- @app.patch()
- @app.trace()

Step – 4 : define the path operation function

- from fastapi import FastAPI
- app = FastAPI()
- @app.get("/")
- async def root():
- return {"message": "Hello World"}

- This is our "path operation function":
- **path**: is /.
- **operation**: is get.
- **function**: is the python function below the "decorator" (below @app.get("/")).
- The python function will be called by FastAPI whenever it receives a request to the URL "/" using a GET operation.

Step – 4 : Info

- This python function will be called by FastAPI whenever it receives a request to the URL "/" using a GET operation.
- In this example, it is an async function.
- You could also define it as a normal function instead of async def:
- **Asycn vs normal :** https://fastapi.tiangolo.com/async/#technical-details

Step – 5 : return the content

- from fastapi import FastAPI
- app = FastAPI()
- @app.get("/")
- async def root():
- return {"message": "Hello World"}

- We can return a dict, list, singular values as str, int, etc.
- We can also return Pydantic models.
- There are many other objects and models that will be automatically converted to JSON (including ORMs, etc). Try using your favorite ones, it's highly probable that they are already supported.

Key Takeaways

- Import FastAPI.
- Create an app instance.
- Write a path operation decorator using decorators like @app.get("/").
- Define a path operation function; for example, def root(): ....
- Run the development server using the command fastapi dev.
- Optionally deploy your app with fastapi deploy.